

this list are often 'bread', the mutable list ['bread', 'bread'] is used as a default argument:

```
>>> def addIngredient(ingredient, sandwich=['bread', 'bread']):
...     sandwich.insert(1, ingredient)
...     return sandwich
...
>>> mySandwich = addIngredient('avocado')
>>> mySandwich
['bread', 'avocado', 'bread']
```

But using a mutable object, such as a list like ['bread', 'bread'], for the default argument has a subtle problem: the list is created when the function's `def` statement executes, not each time the function is called. This means that only one ['bread', 'bread'] list object gets created, because we only *define* the `addIngredient()` function once. But each function *call* to `addIngredient()` will be reusing this list. This leads to unexpected behavior, like the following:

```
>>> mySandwich = addIngredient('avocado')
>>> mySandwich
['bread', 'avocado', 'bread']
>>> anotherSandwich = addIngredient('lettuce')
>>> anotherSandwich
['bread', 'lettuce', 'avocado', 'bread']
```

Because `addIngredient('lettuce')` ends up using the same default argument list as the previous calls, which already had 'avocado' added to it, instead of ['bread', 'lettuce', 'bread'] the function returns ['bread', 'lettuce', 'avocado', 'bread']. The 'avocado' string appears again because the list for the `sandwich` parameter is the same as the last function call. Only one ['bread', 'bread'] list was created, because the function's `def` statement only executes once, not each time the function is called. A visualization of the execution of this code is at <https://autbor.com/sandwich>.

If you need to use a list or dictionary as a default argument, the pythonic solution is to set the default argument to `None`. Then have code that checks for this and supplies a new list or dictionary whenever the function is called. This ensures that the function creates a new mutable object *each*